

1. Framework para escalar SCRUM / Nexus
2. Filosofía Lean - Método Kanban en el contexto del desarrollo de software
3. Métricas en los 3 enfoques (Tradicional / Lean / Agile)

Unidad 3: Gestión de software como producto

4. Prácticas continuas

Integración continua

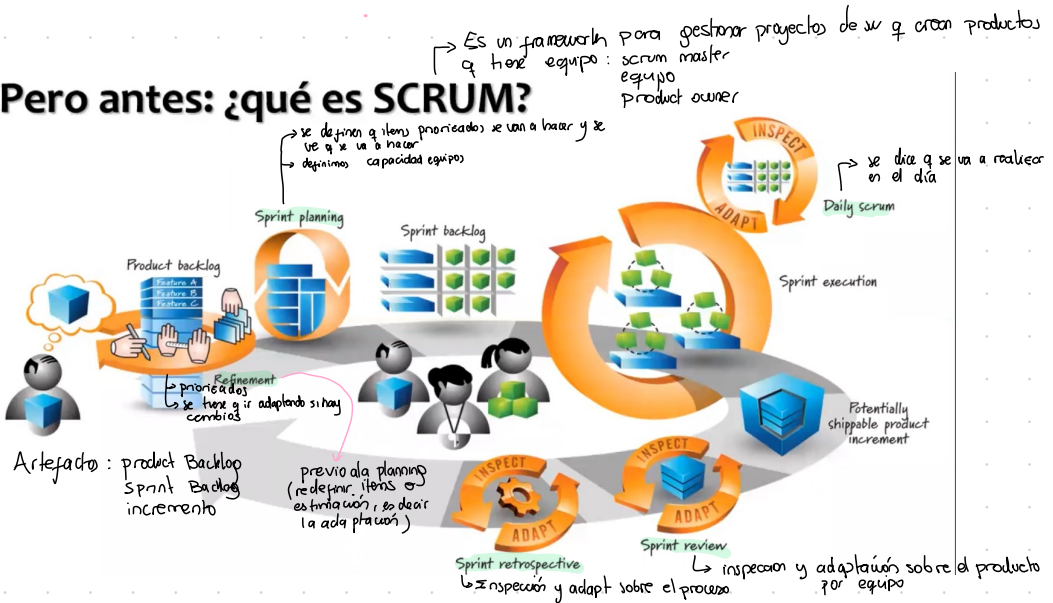
Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

5. Conceptos generales sobre calidad.
6. Importancia de trabajar para y con Calidad. Ventajas y Desventajas.
7. Actividades relacionadas con el Aseguramiento de la Calidad del Software.
8. Principales Modelos de Calidad existentes (CMMI - SPICE - ISO) y sus métodos de evaluación.
9. Diferentes tipos de Auditorías: Auditorías de Proyecto y Auditorías al Grupo de Calidad.
10. Proceso de Auditorías: Responsabilidades. Preparación y ejecución. Reporte y seguimiento.
11. Calidad de Producto:
 - o Planificación de pruebas para el software- Niveles y tipos de pruebas para el software.
 - o Técnicas y herramientas para probar software.
 - o Testing agile
 - o Técnicas y Herramientas para la realización de revisiones técnicas del software.

9 teorías
2 prácticas

Ceremonias

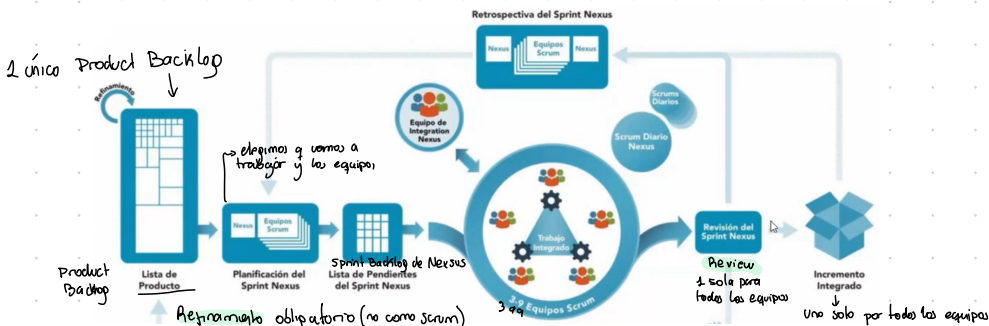
Pero antes: ¿qué es SCRUM?



durante el sprint no me puedes cambiar nada. Los cambios son bienvenidos pero no es el sprint

PQ escalamos?

quiero trabajar en el contexto de SCRUM pero con un equipo más grande



1. Framework para Escalar Scrum

Existen diferentes frameworks para escalar como Nexus, scale scrum, less, SAFe.

Nexus es un Framework que consiste en roles, eventos, artefactos y técnicas que vinculan el trabajo de aproximadamente tres a nueve equipos Scrum, que trabaja sobre un único Product Backlog común para la construcción de un incremento integrado de producto "Terminado". Con un solo Product Owner para entregar un único producto.

Backlog. Esta complejidad está dada por las siguientes dependencias entre equipos: *Debido a que*

1. Requerimientos: el alcance de estos puede superponerse. La forma en que se implementan también puede afectar a los demás.
2. Conocimiento del dominio: el conocimiento del sistema de negocio debería mapearse a los Equipos Scrum para minimizar las interrupciones entre los mismos durante el Sprint.

Responsabilidades (→ Resuelve los aspectos de integración y dependencias q surgen de q muchos equipos estén trabajando en un solo flujo de trabajo para obtener un único incremento)

- **Nexus Integration Team:** Responsable de asegurar que se produzca un incremento integrado a fin de semana en cada sprint. La integración incluye abordar las restricciones técnicas y no técnicas del equipo multifuncional que pueden impedir la capacidad de un Nexus para entregar constantemente un incremento integrado. **Compuesto por: PO + Scrum Master + 2 o más NIT members**

- **Product Owner:** Es responsable de maximizar el valor del producto y el trabajo realizado e integrado por los Scrum Teams en un Nexus, así mismo también es responsable de la gestión eficaz del Product Backlog.

- **Scrum Master:** responsable de asegurar que el marco de trabajo Nexus se entienda y se promulgue como se describe en la Guía de Nexus. Puede ser un Scrum Master en uno o más de los scrums teams en el Nexus.

- Uno o más miembros del Nexus Integration Team: a menudo está formado por miembros de los Scrum Teams que ayudan a los Scrum Teams a adoptar herramientas y prácticas que contribuyen a mejorar la capacidad de los Scrum Teams para entregar un Integrated Increment útil y de valor que cumple con la Definición de Terminado.

Nexus agrega o extiende los eventos definidos por Scrum. La duración de los eventos Nexus se guía por la duración de los eventos correspondientes en la Guía de Scrum. Tienen definido un bloque de tiempo adicional a sus correspondientes eventos de Scrum.

- **Nexus Sprint:** Los scrums teams producen un solo Integrated Increment (Incremento de integración), es lo mismo que scrum.

- **Refinamiento entre equipos:** El Refinamiento Entre Equipos del trabajo del Product Backlog reduce o elimina las dependencias entre equipos dentro de un

Nexus. El Product Backlog debe descomponerse para que las dependencias sean transparentes, se identifiquen entre equipos y se eliminen o se minimicen.

El Refinamiento Entre Equipos del Product Backlog a escala sirve a un propósito dual:

o Ayuda a los Scrum Teams a prever qué equipo entregará qué elementos del Product Backlog.

o Identifica dependencias entre estos equipos.

El Refinamiento Entre Equipos es continuo. La frecuencia, duración y participación del Refinamiento Entre Equipos varía para optimizar estos dos propósitos.

- **Nexus Sprint Planning:** El propósito de la Nexus Sprint Planning es coordinar las actividades de todos los Scrum

Teams dentro de un Nexus para un solo Sprint. Los representantes apropiados de cada Scrum Team y el Product Owner se reúnen para planificar el Sprint.

Resultados:

o Objetivo de Sprint para cada Scrum team.

o Objetivo de Nexus.

- o Un Nexus sprint backlog.

- o Un Sprint backlog.

- **Nexus Daily Scrum:** foco $\rightarrow$ Integración y dependencia

o Se discuten problemas de integración y se inspecciona el progreso hacia el objetivo de sprint del nexus.

o Solo asisten representantes de cada Scrum team.

o La Daily Scrum de cada Scrum Team complementa la Nexus Daily Scrum creando planes para el día, centrados principalmente en abordar los problemas de integración planteados durante la Nexus Daily Scrum.

- Daily Scrum → q es lo q se hizo, q se va a hacer e impedimentos. (1° enle)

Refinamato - Nexus sprint - Nexus sprint - Nexus Daily - Nexus sprint - Nexus sprint
planning Scrum Review retrospectives

Daily Scrum

Los Scrum teams no solo se reúnen o coordinan en el Nexus daily scrum, sino que durante el día, puede existir comunicación entre equipos con mayor detalle sobre adaptar o replanificar el resto del trabajo del sprint.

• **Nexus Sprint Review:** La Nexus Sprint Review se lleva a cabo al final del Sprint para brindar retroalimentación sobre el **Integrated Increment** terminado que el Nexus ha construido durante el Sprint y determinar adaptaciones futuras.

La Nexus sprint review reemplaza a las sprint reviews de cada scrum team.

• **Nexus Sprint Retrospective:** El propósito de la Nexus Sprint Retrospective es planificar formas de mejorar la calidad y la eficacia en todo el Nexus. El Nexus inspecciona cómo fue el último Sprint con respecto a individuos, equipos, interacciones, procesos, herramientas y su Definición de Terminado. Además de las mejoras de cada equipo, las Retrospectivas de los Scrum Teams complementan la Nexus Sprint Retrospective mediante el uso de inteligencia de abajo hacia arriba para centrarse en los problemas que afectan al Nexus en su conjunto.

→ los representantes de **NIT** identifican los problemas q impactan en + de un equipo y después c/ equipo hace su propia retro y también se ven estos temas de rel y dependencia. Luego los representantes de equipo se reúnen y trabajan en la retrospectiva Nexus

Artefactos

Representan trabajo o valor y están diseñados para maximizar la transparencia. El Nexus Integration Team trabaja con los Scrum Teams dentro de un Nexus para garantizar que se logre la transparencia en todos los artefactos y que el estado del Integrated Increment se entienda.

Cada artefacto contiene un compromiso, y estos existen para reforzar el empirismo y el valor de Scrum para el Nexus y sus interesados.

• **Product Backlog:** Hay uno solo que contiene la lista de todo lo que el Nexus y sus Scrum Teams necesitan para mejorar el producto. El Product Backlog debe entenderse con tal nivel que permita detectar y minimizar las dependencias. Es responsabilidad del Product Owner. El compromiso es el Objetivo del Producto, que describe el estado futuro del producto y sirve como un objetivo a largo plazo para el Nexus.

• **Nexus Sprint Backlog:** está compuesto del Objetivo del Sprint de Nexus y elementos del Product Backlog de cada Scrum Team del Nexus. Se usa para resaltar las dependencias y el flujo de trabajo durante el Sprint, y se actualiza durante el mismo a medida que se obtiene más conocimiento. Este debe tener un nivel de detalle tal que permita que Nexus pueda inspeccionar su progreso en la Nexus Daily Scrum. → q le va a hacer a c/ equipo reducir en el sprint Su compromiso es el Objetivo de Sprint de Nexus, que es un único objetivo para Nexus. Es la suma de todo el trabajo y los Objetivos de Sprint de los Scrum Teams dentro del Nexus. Se crea en la Nexus Sprint Planning y se agrega al Sprint Backlog de Nexus. Los Scrum Teams lo tienen en cuenta a medida que trabajan. En la Nexus Sprint Review se debe de mostrar la funcionalidad de valor y útil que está terminada para alcanzar el Objetivo del Sprint.

• **Integrated Increment:** es la suma actual de todo el trabajo integrado completado por un Nexus en relación con el Objetivo del Producto. Se inspecciona en la Nexus Sprint Review pero puede entregarse antes de la misma. Debe cumplir con la definición de terminado.

El compromiso de este artefacto es la Definición de Terminado, que define el estado del trabajo integrado cuando cumple con la calidad y las medidas requeridas para el producto. El incremento se termina sólo cuando está integrado, es de valor y utilizable. Es responsabilidad del Nexus Integration team una definición de terminado que se pueda aplicar en cada sprint, y todos los Scrum Team deben definir y adherirse a esta definición. Cada Scrum Team se autogestiona para llegar a este estado, pudiendo aplicar una definición de terminado más estricta pero nunca usar criterios menos rigurosos de lo acordado para el Integrated Increment.

Filosofía LEAN y KANBAN

Filosofía Lean

Es una filosofía o enfoque más viejo que ágil, que nació en Japón ('40) con Toyota, y busca maximizar el valor generado al cliente con el mínimo consumo de recursos, argumentando que todo el esfuerzo que no cree valor, se considera desperdicio. Esto permite reducir los esfuerzos en crear artefactos que no agreguen valor a un producto, concentrando los esfuerzos en los aspectos esenciales.

La filosofía "Lean", conduce a una visión integrada de la cultura y la estrategia para atender al cliente final con alta calidad, bajo costo y tiempo de entrega, produciendo exactamente lo que el cliente final quiere, cuando lo quiere, dónde lo quiere, a un costo mínimo y precio justo. Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.

Para esto utiliza el método Just in Time, el cual permite el intercambio del producto entre fases del proceso de desarrollo o a la entrega del producto final al cliente justo a tiempo, esto es poco antes de que lo usen y en la cantidad justa y necesaria, "analice cuando lo necesite, no antes".

7 Principios Lean

1. Eliminar desperdicios

Es quizás el más importante. Debemos tener un proceso que sea eficiente y que sume valor en todos los pasos del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio.

Todo esto no quiere decir que el proceso es perfecto, porque según todas las filosofías de mejora continua (como esta o ágil) marcan que siempre se puede mejorar el proceso.

Lo que se busca evitar:

- Producir funcionalidades en exceso, que luego no se usan
- Retrabajo.

Las 7 desperdicios de Lean

Se pondrán los desperdicios para software y entre corchetes para la industria manufacturera.

- Características extras [producir productos extras]

No solo tiene que ver con las funcionalidades que se desarrollan demás, sino también con Up Front Specification (especificar anticipadamente), porque cuando se intenta satisfacer esos requerimientos en etapas posteriores de implementación, la información está obsoleta, incompleta o errónea, debido a que cuando se especificó el requerimiento había incertidumbre o falta de información.

Esto se relaciona con el principio lean 4 (diferir compromisos hasta el último momento responsable).

- Trabajo a medias [acumulación de stock]

Esto representa el trabajo que se realiza a medias (US, casos de uso, bugs, etc.) sin terminar, lo cual genera que deba ser retomado posteriormente para su finalización. Esto se relaciona con el desperdicio de Cambio de Tareas.

- Proceso extra

Esto se refiere a los pasos extras que se ejecutan en el proceso de desarrollo que no aportan valor al producto. El objetivo es identificarlos y eliminarlos, para no destinar esfuerzo a actividades que no aportan valor.

- Búsqueda de información [versiones, etc]

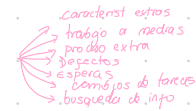
Esto termina siendo un problema importante y difícil de manejar si no se implementa la disciplina SCM, porque no contamos con información suficiente y/o de fácil acceso para tener una trazabilidad sobre los ítems de configuración, lo que dificulta identificar el impacto de los cambios y ralentiza la resolución de errores.

- Defectos

Son errores que se trasladan de una etapa a otra etapa posterior en la cual se introdujo. Esto provoca retrabajo. Un antídoto a este problema es la disciplina de Aseguramiento de Calidad, que permite evitar esos errores a lo largo del desarrollo del producto.

- Esperas [costos de logística]

Esto es el tiempo de espera que una persona tiene que pasar para poder realizar su trabajo. Ese tiempo de espera puede ser causado porque depende del trabajo de otra persona, porque requiere aprobación de un superior, etc.



identificar req anticipadamente y reducirlos ya q después que da no sirven

→ organizar la info

• Cambios de tareas (Multitasking) → cuando una persona cambia constantemente de tarea su productividad cae.
Como el desarrollo de software es un trabajo intelectual, uno de los desperdicios más grandes al hacer este tipo de trabajo, es el cambio de tareas o multitasking. Esto genera una pérdida de tiempo en prepararnos para realizar cada tarea.
Por eso en Kanban se hace uso de limitar el WIP (Work in Progress).

2: Amplificar el aprendizaje

Tiene que ver con la transparencia y transformación de conocimiento implícito en explícito, lo cual fortalece

al equipo de trabajo, donde los individuos intercambien sus experiencias y conocimientos para contribuir con el aprendizaje colectivo. Por otro lado, todo conocimiento que se genere debe compartirse para que sea fácilmente accedido por toda la organización.

3: Mantener la integridad conceptual → Asegurar q el producto sea consistente, consistente y de alta calidad

La calidad del producto no se negocia y la simplicidad es un factor clave (principios Agile): el producto tiene que satisfacer las expectativas y necesidades del cliente, esa es la principal prioridad y cada decisión se alinea con esto, no importa si el equipo por momentos se enfoca en otras cosas, no se debe perder de vista que esa es la prioridad principal.

4: Diferir compromisos hasta el último momento responsable

Esto apunta a postergar la toma de decisiones lo suficientemente como para poder tener la mayor información necesaria para tomar esa decisión, pero a la vez, antes de que ese momento sea muy tarde.

5: Dar poder al equipo

Otorgar libertad de acción y poder de decisión al equipo. Para ello, el equipo debe ser multifuncional y autogestionado, y sus miembros deben estar capacitados y motivados.

6: Ver el todo → entender el producto y el valor q ese producto le da a clar al cliente

En Lean se busca tener una visión holística, que permita asociar y comprender el todo, el producto, el valor agregado que hay detrás, el servicio que brinda el complemento de los productos, más allá de los objetivos particulares por áreas o gerencias, porque la idea es que los objetivos particulares de las partes estén alineados con los objetivos globales.

7: Entregar lo antes posible

La idea es entregarle al cliente software funcionando lo más rápido posible y que este producto sea útil para él. Se habla de entregarle software antes de que las necesidades de negocio cambien, porque el ambiente donde se desempeña el mismo cambia.

Para esto Lean propone acotar ciclos de desarrollo y entregar rápidamente incrementos pequeños de valor, lo que permite salir pronto al mercado con un producto mínimo que sea viable.

Relación Lean-Ágil

Lean es previo al Manifiesto Ágil. Algunos principios de Agile heredan de los fundamentos de Lean. Ambos están orientados al cliente, a proveer el máximo valor posible. Para esto, entienden que los individuos de los equipos deben estar motivados, de forma tal que la sinergia de este facilite el desarrollo del proyecto. Esta sinergia, se interpreta como un valor agregado para el cliente. La flexibilidad para adaptarse a los cambios y ofrecer valor al cliente, es también un denominador común de ambos enfoques. Otro aspecto que tienen en común es generar productos de calidad y de mejora continua.

Framework para aplicar estos principios lean → Kanban

Definición

- Un proceso de desarrollo de software.
- Una metodología de administración de proyectos.

Kanban es un enfoque para la gestión de formas de trabajo (procesos) para obtener mejora continua.

Esto lo logra a través del principio de "empezar por donde estés", es decir, no plantea introducir cambios revolucionarios, sino que introduce mejoras graduales a un proceso de desarrollo de software, fomentando la evolución gradual de los procesos existentes.

Esta basado en limitar el trabajo en progreso (WIP)

Kanban aprovecha los principios de Lean:

- Definiendo el valor desde la perspectiva del cliente.
- Limitando el trabajo en proceso WIP.
- Identificando y eliminando desperdicios.
- Identificando y eliminando las barreras en el flujo, es decir, todo lo que atrasaría el proceso: Relacionado con el principio de lograr entregar lo antes posible.
- Cultura de mejora continua.

Principios

Principios de Kanban

Visualización del trabajo → c/uno configura el tablero como quiere

El método Kanban utiliza un mecanismo de señalización para hacer visible el trabajo que es requerido por el cliente.

Para ello, divide el trabajo en piezas de trabajo (que pueden ser U.S., temas, épicas, cambios, etc.) y las escribe en tarjetas señalizadoras (kan-bans) que serán ubicadas en tableros kanban.

Estas tarjetas permiten indicar cuando el trabajo se puede "pullar" para realizar un trabajo determinado, además de señalar en qué parte del proceso se encuentra.

El tablero kanban representa un sistema de flujo en el que las piezas de trabajo fluyen a través de las diversas etapas de un proceso, de izquierda a derecha. Cada etapa es representada por una columna del tablero, sobre las cuales se aplica la teoría de colas.

Se logra transparencia al hacer visible para todo el equipo el trabajo mediante un tablero kanban siempre disponible y a la vista.

Existen dos tipos de columnas:

- Producción: piezas sobre las que se está trabajando.
- Acumulación: piezas que están listas para pasar a la siguiente etapa (sistema de arrastre).

Limitar el WIP (Work in Progress) →

Se deben asignar límites concretos a cuántas piezas pueden estar en progreso en cada etapa del flujo de trabajo (en cada columna), para evitar cuellos de botella.

Las políticas para limitar el WIP crean un sistema de arrastre (pull): el trabajo es "arrastrado" al sistema cuando otro de los trabajos es completado y queda capacidad disponible, en lugar de "empujar" estos trabajos al paso siguiente cuando hay nuevo trabajo demandado.

Tener demasiado trabajo no finalizado es un desperdicio de tiempo, de dinero y alarga los tiempos de entrega.

Observar, limitar y optimizar la cantidad de trabajo en progreso es esencial para tener éxito con Kanban, consiguiendo mejorar la calidad y el tiempo de entrega de servicio y aumentar la tasa de entrega.

Gestionar el flujo de trabajo

El trabajo fluye sobre un tablero permanente. El tablero puede ser compartido con otros proyectos y otros equipos, ya que se trabaja cada pieza de trabajo de forma individual. Al flujo que se observa en ese tablero, se lo debe analizar, identificando diferentes características como: cuellos de botella, recursos ociosos, tiempos de entrega, tiempos de espera, etc.

→ para mejorar procesos de desarrollo de SW

↳ trabaja con just in time
↳ administración de colas

Visualizar trabajo
limitar WIP
Gestionar flujo de trabajo
Hacer explícitas las políticas
mejorar y evolucionar
ciclos de retroalimentación

No superar el límite
de WIP cuando se
trabaja algo para trabajar

Hacer explícitas las políticas

Las políticas de proceso deben ser escasas, simples, estar bien definidas, visibles, deben aplicarse siempre, y tienen que ser fácilmente modificables. Es una buena práctica poder cambiar fácilmente las políticas, ya que si producen efectos contraproducentes para nuestro proceso o también se considera que no pueden aplicarse las mismas, deben poder cambiarse.

También es importante visualizar las políticas; por ejemplo, colocando resúmenes entre las columnas donde se describe lo que debe estar hecho antes de que una tarjeta se mueva de una columna a la siguiente. También se suele colocar el WIP de cada columna.

Las políticas de calidad o DoD deben estar definidas, publicadas y promovidas para lograr no sólo que se cumplan, si no buscar mejorar continuamente.

Mejorar y evolucionar

Kanban propone una cultura de mejora continua, donde no introduce cambios significativos en los procesos de desarrollo, sino que identifica ese proceso, lo visualiza (hacer explícito para todos los miembros) y propone mejoras sobre él, las cuales se van aplicando de forma gradual a lo largo del tiempo.

Círculo de retroalimentación → reuniones de reunión son avanzadas

Son una parte esencial para cualquier proceso controlado que nos ayuda a realizar cambios evolutivos. Se debe definir con qué frecuencia se realizan las reuniones y donde, depende en qué contexto se presentan ya que es importante para el resultado.

Si se realizan revisiones demasiadas frecuentes pueden obligar a cambiar cosas que no se vieron con los cambios anteriores, pero si no son demasiadas frecuentes, existe un bajo rendimiento durante mucho tiempo.

Asignación de tareas → se ve reflejada con un avatar personalizado

¿Cómo aplicar Kanban?

1. Empezar con lo que se tiene: entender el proceso de desarrollo actual que se está utilizando.
2. Identificar las unidades de trabajo a utilizar (US, CU, defectos).
3. Identificar clases de servicio: diferentes trabajos tienen distintas políticas para tratarlos (DoD). Por ejemplo: requerimientos, defectos, desarrollo y solicitudes.
4. Visualizar el flujo de trabajo diseñando un tablero representando el flujo de trabajo a través de columnas.
5. Definir políticas para cada clase de servicio identificada.
6. Identificar cuellos de botella y resolverlos (asignando más recursos o ajustando el WIP donde corresponda).

Métricas de Kanban

Las métricas más representativas de Kanban son:

- Cycle Time: tiempo de ciclo
- Lead Time: tiempo de entrega
- Touch Time: tiempo de interacción
- Eficiencia del Ciclo de Proceso.

Estas métricas están desarrolladas en el tema de métricas de la unidad 2 de este apunte.

Valores de Kanban

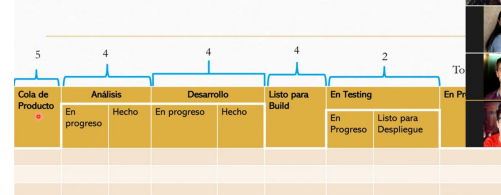
Los valores de Kanban se podrían resumir en una sola palabra, "respeto". Sin embargo, es importante desglosar esto en una serie de nueve valores.

- **Transparencia:** La creencia de que compartir información abiertamente mejora el flujo de valor de negocio.
- **Equilibrio:** El entendimiento de que los diferentes aspectos, puntos de vista y capacidades deben ser equilibrados para conseguir efectividad.
- **Colaboración:** Trabajar juntos. El Método Kanban fue formulado para mejorar la manera en que las personas trabajan juntas, por ello, la colaboración está en su corazón.
- **Foco en el cliente:** Conociendo el objetivo para el sistema. Los clientes en este contexto son externos al servicio, pero pueden ser internos o externos a la organización como un todo. Los clientes y el valor que estos reciben es el foco natural en Kanban.
- **Flujo:** La realización de ese trabajo es el flujo de valor. Ver el flujo es un punto de partida esencial en el uso de Kanban.
- **Liderazgo:** La habilidad de inspirar a otros a la acción a través del ejemplo, de las palabras y la reflexión.
- **Entendimiento:** Principalmente conocimiento de sí mismo (tanto individual como de la organización) para ir hacia adelante. Kanban es un método de mejora, por lo que conocer el punto de inicio es la base de todo.
- **Acuerdo:** El compromiso de avanzar juntos hacia los objetivos, respetando las diferencias de opinión. Esto no es gestión por consenso sino un compromiso dinámico para mejorar.
- **Respeto:** Valorando, entendiendo y mostrando consideración por las personas. De manera apropiada al pie de esta lista se encuentra la base sobre la cual reposan el resto de los valores.

Principios directores

1. Sostenibilidad: relativo a encontrar un foco sostenible y un foco en la mejora.
2. Orientación al servicio: enfocado a conseguir rendimiento y satisfacción del cliente.
3. Supervivencia: relativa al mantenimiento de la competitividad y adaptabilidad.

Definir el WIP...



1. Entender el proceso de desarrollo actual
2. Identificar unidades de trabajo (U.S., temas, etc.)
3. Identificar clases de servicio
4. Revisar el flujo de trabajo realizando un tablero
5. Definir políticas para los clases de servicio
6. Identificar cuellos de botella y resolverlos

- Empezar con lo q se tiene
- Entender el proceso actual
- Acordar límites WIP para c/etapa
- Comienza a fluir el trabajo

¿Cómo aplicar Kanban en nuestro proyecto?

- Proceso: modelar nuestro proceso.
- Trabajo: decidir la unidad de trabajo.
- Límites de WIP: limitar el WIP para ayudar al flujo de trabajo.
- Política: definir políticas de calidad.
- Cuellos de Botella y Flujo: mover recursos a los cuellos de botella.
- Clase de Servicio: diferentes trabajos tienen diferentes políticas - definición de hecho ("done"), para cada estado.
- Cadencia: Releases, planificaciones, revisiones

Expreso. Los demás trabajos se ponen en espera. A riesgo: q se permitan mas de una.

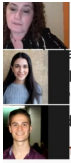
Fecha Fija. Se elige cuando trabajarla en función de esa fecha fija.

Estándar: situación normal para lo q armamos el tablero.

Intangible: trabajo q cuenta y el cliente lo pedia visualizar, pero está bueno hacerlo. (Migración)

Métricas → para medir el proceso de q tenemos buenos somos para generar valor

Cycle Time (Tiempo de ciclo)



Es la métrica que registra el tiempo que sucede entre el inicio y el final del proceso, para un ítem de trabajo dado. Se suele medir en días de trabajo o esfuerzo.

Medición más mecánica de la capacidad del proceso

tiempo de Terminación

→ Desde q se empieza a trabajar en una tarea hasta q se pasa

Lead Time (Tiempo de entrega)

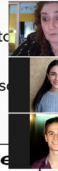
- Es la métrica que registra el tiempo que sucede entre el momento en el cual se está pidiendo un ítem de trabajo y el momento de su entrega (el final del proceso). Se suele medir en días de trabajo.

- Ritmo de entrega

→ Desde q se solicita el trabajo hasta q se entrega

Touch Time (Tiempo de Tocado)

- El tiempo en el cual un ítem de trabajo fue realmente trabajado (o "tocado") por el equipo. *cuanto tiempo efectivo se trabajó*
- Cuántos días hábiles pasó este ítem en columnas de "trabajo en curso", "oposición con columnas de cola / buffer y estado bloqueado o sin trabajo del equipo sobre el mismo."



$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

métricas cambian

Eficiencia del Ciclo de Proceso

$$\% \text{ Eficiencia ciclo proceso} = \text{Touch Time} / \text{Lead Time}$$

Lead

Elapse

→ mientras + cercano a 1 mas eficiente

métricas en los 3 enfoques

→ medida numérica para evaluar algo

Una métrica es un número que representa el grado o presencia de un determinado conjunto de atributos respecto de lo que se quiere medir (proceso, producto o proyecto), expresadas de tal forma que permitan una medición objetiva de la realidad.

Todas las métricas poseen un costo asociado, debido a que se deben destinar recursos para su planificación, ejecución y análisis, y no siempre es factible disponer de esos recursos en un proyecto. Esto implica un análisis de costo-beneficio para evitar definir métricas que no aporten un valor a la organización/equipo/proyecto/etc. Las métricas no sólo hay que calcularlas y dejarlas, hay que utilizarlas para que justifiquen ese costo.

Las métricas no son para evaluar a la gente. No deben ser utilizadas para castigar o beneficiar a la gente, ya que es posible que las personas comiencen a fingir en el valor de las métricas para lograr objetivos finales, lo cual resulta perjudicial para el análisis del fenómeno que se desea medir.

¿Para qué medir?

- Para señalar, controlar y supervisar el desarrollo del proyecto.
- Para predecir al estimar proyectos de software.
- Para evaluar, es decir, para tener una noción de los costos, por ejemplo.
- Para mejorar el proceso, el proyecto o el producto.

obtener predictibilidad

ADICIONAL

Dominio de métricas

métrica tiene q ser un número

Las métricas se dividen en tres dominios, el cual, cada uno posee un foco distinto de medición:

→ enfocada al proceso de desarrollo, mejorar el proceso detectando puntos débiles.

- Métricas de proceso:** son métricas estratégicas a nivel organizacional, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos de software a largo plazo. Son públicas.

Y Son responsabilidad del Ingeniero de Procesos. Toma métricas de proyecto y despersonalizarla

Los indicadores de proceso permiten tener una visión profunda de la eficacia de un proceso, determinando qué funciona de acuerdo a lo esperado y qué no. Proporcionan beneficios significativos a medida que la organización trabaja para mejorar su nivel global de madurez del proceso.

- Métricas de proyecto:** están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo. Son responsabilidad del Líder de Proyecto y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas. Estas métricas son privadas de ese proyecto y solo son visibles para los involucrados en el mismo.

optimizando recursos

→ mide los recursos para ayudar a planificar y mejorar el proyecto

Se utilizan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzcan riesgos potenciales y por lo tanto, problemas.

Además, se utilizan para evaluar la calidad de los productos en todo momento y en caso de ser necesario, modificar el enfoque para mejorar la calidad, minimizando defectos, retrabajo y por ende el costo total del proyecto. *Relacionadas a la triple restricción (esfuerzo, Alcance y tiempo)*

- Métricas de Producto:** están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo y de Testing y son particulares de ese producto.

Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la

efectividad de los casos de prueba y calidad del software a construir.

Se deben controlar los artefactos resultantes del proceso de desarrollo (componentes y modelos) para

miden los costos del sw para evaluar la calidad del producto

garantizar:

- Que cumplan con los requerimientos del cliente;
- Que cumplan con los requerimientos de calidad;
- Que estén libre de errores;
- Que se realizaron bajo los procedimientos de calidad.

Métricas básicas

- Tamaño del producto → contamos los c.v o líneas de código → producto
- Esfuerzo → trabajo (horas personas lineales) } proceso
- Tiempo (calendario)
- Defectos → producto

Regla de oro ágil sobre Métrica

Medir la q sea necesario y nada mas

2 principios q guían las métricas

Nuestra mayor prioridad es satisfacer al cliente...
el sw funcionando en la princ medida de progreso

Métricas en el enfoque tradicional

En el enfoque tradicional se hace énfasis en los 3 dominios arriba mencionado.

Cuando se planifican los proyectos, allí se definen qué métricas se utilizarán, quiénes serán responsables de estas, cómo se calcularán, etc.

No todas las métricas le interesan a todo el mundo, dependiendo del momento y rol es lo que interesa medir.

Por ejemplo:

- Testers: les interesa el esfuerzo y cantidad de defectos encontrados principalmente, porque es lo que más afecta a este rol.
- Líder de proyecto: El enfoque de las métricas estará más relacionado al calendario, los costos, plazos de tiempo a cumplir, etc. por el contrato que se tiene con el cliente, y la relación entre el esfuerzo y el tiempo.

Ágil

Ágil hace poco en las métricas de producto
velocidad no se estima, capacidad si.

Métricas en ambientes ágiles

Las métricas sirven para un equipo y no son extrapolables por lo que dice el agilismo a otros proyectos o equipos. En ágil hay un principio que habla específicamente de métricas (la mejor métrica de progreso es el software funcionando).

Este principio apunta a que vamos a medir producto (no proceso ni proyecto). Solo se debe medir lo que sea necesario y nada más, es decir, lo que agregue valor para el cliente.

Métrica de velocidad → se mide al final del sprint, mide la us terminados y aceptados por el P.O. (tareas incompleta no)

Es la métrica más importante del enfoque (métrica de producto), y mide la cantidad de puntos de historia que se realizaron en un Sprint y fueron aceptados por el Product Owner. Hay que recordar que no se cuentan las historias parcialmente terminadas, sólo las que están completadas y aceptadas por el PO. Esta métrica se calcula (no se estima) luego de que el PO. acepte la implementación de una US. suele representarse con un gráfico de barras para medir la estabilidad del equipo.

Esta métrica, y sus valores a lo largo de un proyecto ágil, permiten analizar si el equipo posee una estabilidad a lo largo del mismo, lo cual también se relaciona con un principio del manifiesto ágil (desarrollo sostenible).

Métrica de capacidad → se usa para planificar cuánto U.S. se pueden hacer en el prox sprint

Es una métrica de proyecto que mide el compromiso de un equipo para un determinado sprint, en horas de trabajo ideales. Es por esto que, se utiliza para planificar, ya que permite definir cuántas historias de usuario se van a tomar del Product Backlog para implementar en el próximo sprint.

Se estima al principio de un sprint, por lo que también se la llama velocidad estimada. Se puede estimar en horas ideales o en Story Points.

Running Tested Features (RTF) → mide la cant de func terminados y probados q están funcionando, evalúa el progreso real del product

Mide la cantidad de features testeadas que están funcionando, es decir, cuántas piezas de producto (historias de usuarios, casos de usos, requerimientos) se terminaron y están en ejecución.

El problema de esta métrica es que no tiene en cuenta la complejidad de las piezas de producto que se implementan. Es una medición absoluta.

Métricas Kanban

Métricas de software en Lean (KANBAN)

El foco de medición de Kanban es el proceso, ya que el sistema de trabajo de Kanban es introducir mejoras en un flujo de trabajo continuo

Es decir, se mide el comportamiento del proceso en función al tiempo que se demoran las características.

Lead Time

Métrica de vista o perspectiva del cliente. Es la más importante para el cliente. Mide desde el momento en que el cliente me pide algo (entra al Backlog) hasta que yo se lo entrego.

Cycle Time

Mide el tiempo desde que el equipo comenzó a trabajar sobre una funcionalidad pedida, hasta que se lo entrega, eliminando el tiempo de espera en el Backlog. Por esto se la considera como una vista interna, que sirve al equipo de trabajo y no es tan relevante para el cliente. Es igual al Lead Time menos el tiempo que estuvo en el Backlog.

Touch Time

Mide los tiempos en los cuales las personas están efectivamente trabajando sobre una unidad de trabajo (columnas de producción), sin tener en cuenta aquellos tiempos de espera (columnas de acumulación).

Eficiencia del ciclo de proceso

Esta métrica mide la eficiencia del tiempo para entregar un trabajo, respecto al tiempo destinado a su ejecución. Se calcula como Touch time / Lead time.

Mientras más cercano a 1 sea su valor, más eficiente es el proceso, ya que todo el tiempo se estuvo trabajando sobre alguna funcionalidad, y no hubo muchos momentos de espera en columnas de acumulación.

$$\text{Touch Time} \leq \text{Cycle Time} \leq \text{Lead Time}$$

Respuesta a pregunta de parcial

"Realice un análisis de los principios Lean y explique las prácticas que propone Kanban para aplicar en forma concreta los principios."

- Eliminar desperdicios **V.T → L.WIP**
 - Visualizar el trabajo (permite visualizar el flujo de trabajo y exponer aquellas actividades que no aportan valor y aquellas que generan tiempos de espera que "traban" el flujo de trabajo)
 - Limitar el WIP (el trabajo a medias genera retrabajo, lo cual es un desperdicio. Al limitar el WIP, se evita ese trabajo a medias para concentrar esfuerzos)
- Amplificar el aprendizaje **HEP y ENJOYA**
 - Hacer explícitas las políticas (que todos los miembros tengan acceso a las políticas usadas para cada unidad de trabajo, actividades a realizar, información necesaria del dominio, etc.)
 - Evolución y mejora continua (no introducir cambios grandes, sino que visualizar el proceso de trabajo actual y aplicar mejoras en él)
- Embeber la integridad conceptual
- Postergar decisiones hasta el momento responsable **L.WIP**
 - Limitar el WIP (particularmente el WIP de la columna backlog)
- Ver el todo **V**
 - Visualizar el trabajo (a través del tablero permite que todos los miembros observen todo el trabajo que se hace, el sentido de cada una de las tareas y como estas conforman el flujo de trabajo como un todo que desencadena en la satisfacción de los clientes)
- Dar poder al equipo **HP**
 - Haciendo explícitas las políticas y conformando un sistema de trabajo pull, donde cada miembro toma el trabajo cuando esté en condiciones de realizarlo, y no se lo impone nadie.
- Mejorar **colaborativamente**
- Entregar lo antes posible **GF**
 - Gestionar el flujo de trabajo (permite ver de principio a fin los pasos que se deben seguir para lograr cumplir con las necesidades del cliente y hacer aquellos ajustes que sean necesarios para entregar el mayor valor posible, en el menor tiempo posible)

Int. continua → ir subiendo al código varias veces al día así se ven errores más rápido y esta para ser desplegado

Entrega continua → q el sw este siempre en un estado de entregable versión para q pueda ser puesta en producción

Despliegue continuo → despliegue ocurre automáticamente después de pasar las pruebas si todo está bien el sist despliega la nueva versión

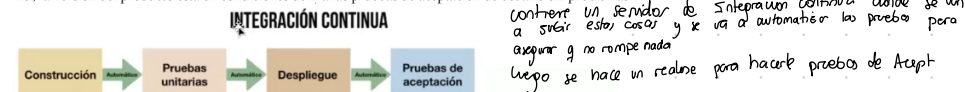
Prácticas continuas

Los desarrolladores van subiendo su código al repo varias veces al día. Se detectan errores más temprano y el sw puede ser desplegado continuamente

Continuous Integration → Integración continua

Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día. Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana. Así resulta más fácil detectar y corregir los errores.

La integración continua es asegurar que el software pueda ser desplegado en cualquier momento, es decir, que el código compile y que sea de calidad. Así, la versión del producto está en condiciones de ir a las pruebas de aceptación de usuario sin problemas.



contiene un servidor de integración continua donde se van a subir este código y se va a automatizar las pruebas para asegurar q no rompa nada luego se hace un release para hacerle pruebas de Acept

Continuous Delivery

Entrega Continua

Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento.

No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, el Product Owner de turno, decide pasar a producción.

hay un componente «humano» a la hora de tomar la decisión. se prioriza que la versión esté en un estado en el que pueda ser puesta en producción.

La integración continua es prerequisite de la entrega continua. Con esto se refiere a que los artefactos producidos en el servidor de integración continua son puestos en producción con solo un click. El software debe estar siempre en un estado de "entregable", es decir, el software buildea, el código se compila y los tests pasan.

ENTREGA CONTINUA



un click o algo así para q empiece
Manual
Automático

Continuous Deployment – Estrategias de Deployments

Despliegue continuo

Consiste en poner en el ambiente de producción del usuario final el producto. A diferencia de la entrega continua, en el despliegue continuo no existe la intervención humana para desplegar el producto en producción. Para esto, se utilizan pipelines que contienen una serie de pasos que deben ejecutarse en un orden determinado para que la instalación sea satisfactoria. **Estrategias de despliegue**

Canary Deployment

En ingeniería de software, Canary deployment es la práctica de realizar lanzamientos por etapas. Primero implementamos una actualización de software para una pequeña parte de los usuarios, para que puedan probarla y proporcionar comentarios. Una vez aceptado el cambio, la actualización se extiende al resto de usuarios.

Blue-Green Deployment Hay 2 entornos: Azul y Verde. Se va transnando el tráfico gradualmente a la nueva, si algo falla se vuelve a la anterior

Es un modelo de lanzamiento de aplicaciones que transfiere gradualmente el tráfico de usuarios de una versión anterior de una aplicación o microservicio a una nueva versión casi idéntica, las cuales se ejecutan en producción.

La versión anterior puede denominarse entorno azul, mientras que la nueva versión puede denominarse entorno verde. Una vez que el tráfico de producción se transfiere por completo de azul a verde, el azul puede quedar en espera en caso de reversión o retirada de producción y se actualiza para convertirse en la plantilla sobre la que se realiza la próxima actualización.

Blue: la versión actual q se está usando
Green: la nueva versión - para a ser Blue cuando ya está listo

DESPLIEGUE CONTINUO



Automático

La diferencia entre estas tres prácticas es el nivel de automatización de las pruebas.

Lo + importante es automatizar las pruebas y q estén bien hechas

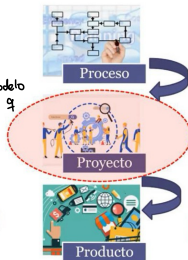
CALIDAD

El PROYECTO genera un PRODUCTO, y se va generando de manera iterativa e incremental. Y los proyectos para generar el producto usan PROCESOS. Basándose en procesos definidos o empíricos.

Analiza el proceso y armar un proyecto cuyo resultado NO es un producto si no un PROCESO MEJORADO, .i. mi producto va a ser mejor

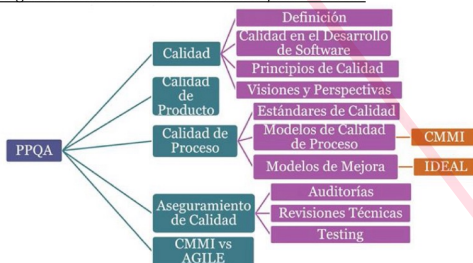
Modelos para crearlos
Modelos para mejorarlos
Modelos para evaluarlos

una vez q apliq el modelo llamo a alguien para q me evalúe



Nos da el contexto para crear un proyecto cuyo resultado va a ser un proceso definido. Es un modelo cíclico que mejora el proceso existente en una organización.

Aseguramiento de calidad de Proceso y de Producto



MODELO IDEAL → Plantea como encarar un proyecto de Mejora de procesos

Inicio → busca un apoyo en la org (sponsors) y comencemos a trabajar. Se identifican necesidades de cambio

Diagnóstico → Donde estoy parado, ¿esta mal, ¿esta bien y a donde queremos ir?

Establecimiento → Se elabora un plan de acción → lo q tenemos q hacer para poder tener tu proceso mejorado

Acción → Ahora se definen los roles, actividades ¿COMO? Se efectúan las cambios y el proceso definido se pone a prueba en algunos proyectos, no en todos y se reúne info

Learning → se evalúan los logros, la info recolectada si sale bien se empieza a usar y aplicárselos a los proyectos

MODELOS DE CALIDAD → como referencia para lograr lineamientos q nuestro proceso tiene q cumplir para llegar a un objetivo CMMi, ISO. Es un modelo usado como ref para crear. Te ayuda a ver q practicas usar en mi proceso

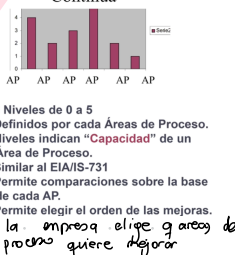
Lo importante es q exista alguien responsable de cubrir las actividades de c/uno de los roles o grupos

CMMI: Representaciones

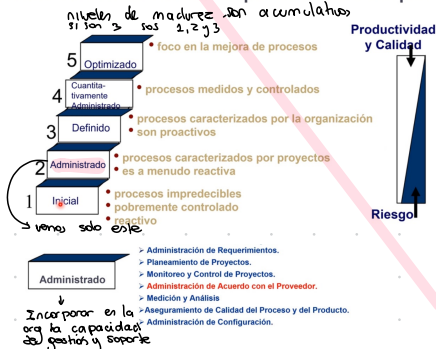
Por Etapas



Continua



CMMI: Niveles de la Representación por Etapas



Modelos de calidad y sus métodos de evaluación

Modelos para la mejora de procesos

La mejora continua de procesos significa comprender los procesos existentes y cambiarlos para incrementar la calidad del producto o reducir los costos y el tiempo de desarrollo.

Los modelos NO te dicen cómo hacer las cosas. Son modelos descriptivos

El propósito de los modelos de mejora es analizar el proceso que tiene la organización y armar un proyecto cuyo resultado, en lugar de ser un producto, es un proceso mejorado que se vuelva de nuevo a la organización con la idea de que se produzca un producto mejor.

Modelo ideal (arriba)

Modelo SPICE (Software Process Improvement Capability Evaluation)

Es un modelo creado para la mejora de procesos software (ISO 15504). Es una adaptación para la evaluación de procesos de desarrollo software por niveles de madurez, dividido en 6 niveles.

Es un modelo dual, teórico. Tiene 2 partes:

- Modelo de Calidad

• SPICE + IDEAL: son ambos modelos de mejora de proceso. Estos modelos de mejora se usan para crear Proyectos de Mejora para una organización. El resultado de este proyecto va a ser un proceso definido que se usará para hacer Proyectos.

Nivel	Estado
Nivel 0 - Organización inmadura	La organización no tiene una implementación efectiva de los procesos
Nivel 1 - Organización básica	La organización implementa y alcanza los objetivos de los procesos
Nivel 2 - Organización gestionada	La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
Nivel 3 - Organización establecida	La organización utiliza procesos adaptados basados en estándares
Nivel 4 - Organización predictiva	La organización gestiona cuantitativamente los procesos
Nivel 5 - Organización optimizada	La organización mejora continuamente los procesos para cumplir los objetivos de negocio

6 niveles de madurez

Modelo de Calidad para evaluar procesos

Plantean una definición teórica (lineamientos) del proceso que se utilizará en una organización (con diferentes niveles de detalle según lo que se necesite), para luego compararlo con la ejecución del proyecto donde se implementa ese proceso de desarrollo. Esto permite medir qué tanto se está cumpliendo en el proyecto con lo que plantea la definición teórica del proceso de desarrollo (definición de roles, asignación de responsabilidades, actividades a realizar, etc.), a través de las auditorías de proyecto.

Existen un montón de modelos de calidad. Los más utilizados en Argentina son el CMMI, CMMI 2.0, etc.

Capability Maturity Model Integration - CMMI

Es un modelo de calidad para la mejora y evaluación de procesos para el desarrollo, mantenimiento y operación de sistemas de software, que constituye un conjunto de buenas prácticas que son publicadas en modelos.

Es un modelo de referencia, son descriptivos. Te dice que tienes que alcanzar determinado objetivo, pero vos definís qué práctica vas a implementar para lograrlo.

Este modelo empírico recopiló buenas prácticas a lo largo de la historia de diferentes organizaciones

El objetivo es que pueda ser usado para guiar la mejora de procesos

Existen 3 constelaciones o modelos de negocio que cubren los modelos de CMMI:

- Desarrollo (CMMI-DEV): provee la guía para medir, monitorear y administrar los procesos de desarrollos de software. Tiene dos representaciones, por etapas y continua.
- Adquisición (CMMI-ACQ): provee la guía para permitir delegar el control de ese proyecto para obtener el producto o hacer cosas, sino que contrata gente que haga las cosas/trabajos por ellos
- Servicios (CMMI-SVC): provee la guía para entregar servicios, externos o internos.

Estas constelaciones poseen los modelos, capacitaciones y toda la información que es necesaria para que las empresas puedan acreditar distintos niveles de madurez.



Áreas de proceso

- Un área de proceso es un conjunto de prácticas relacionadas en una zona que, cuando se implementan en conjunto, satisfacen un conjunto de objetivos considerados importantes para hacer mejoras significativas en el mismo proceso.
- Existen en total 22 áreas de procesos en todos los niveles de madurez

Representaciones CMMI-DEV

- Por etapas: Identificaba a las organizaciones y las dividía en maduras (eran las del nivel del 2 al 5) e inmaduras (nivel 1), mientras más maduras más capacidad de lograr sus objetivos, bajando sus riesgos. La ventaja de esta es que facilita la comparación entre organizaciones. (madurez organizacional).

Niveles que se acreditan en la representación por etapas:

- o Inicial: Las organizaciones en este nivel no disponen de un ambiente estable para el desarrollo y mantenimiento de software. Nivel de las organizaciones inmaduras.
- o Administrado: la organización han asegurado que los requisitos son gestionados y de que los procesos se planifican, se realizan, se miden y controlado. También, hay un razonable seguimiento de la calidad.
- o Definido: la organización ha alcanzado todos los objetivos de niveles de madurez 2 y 3. Aquí los procesos se describen con más detalle y más rigurosidad que en el nivel de madurez 2. También se realiza formación del personal, técnicas de ingeniería más detalladas y un nivel más avanzado de métricas en los procesos.
- o Cuantitativamente administrado: Se caracteriza porque las organizaciones disponen de un conjunto de métricas significativas de calidad y productividad, que se usan de modo sistemático para la toma de decisiones y la gestión de riesgos.
- o Optimizado: este nivel se centra en mejora continua del rendimiento de los procesos a través de los aumentos y mejoras tecnológicas innovadoras.

- Continua: En lugar de medir la madurez de toda la organización, mido la capacidad de un proceso en particular. Procesos de nivel cero son los que no se ejecutan. Apunta a mejorar áreas de proceso que uno elige.

Cuadro comparativo CMMI por Etapas y Continuo

Aspecto	Por etapas	Continuo
Características	Acreditando determinado nivel de madurez de una organización como un todo, dependiendo de qué tantas prácticas realicen, de acuerdo a lo especificado en el modelo CMMI.	Acreditando determinado nivel de madurez de un área de proceso de una organización, dependiendo de qué tantas prácticas realicen concernientes a esa área, de acuerdo a lo que la organización quiera especializar.
Ventajas	Permiten integrar las formas de trabajo de toda la organización. Garantiza cierto nivel de calidad en toda la organización, según el nivel que se acredite. Permite sentar bases en la organización y aumentar los niveles de calidad en toda la empresa.	Permite a una organización especializarse en un área de proceso, independientemente del resto de áreas. La organización elige qué prácticas realizar, de acuerdo al área en la que se especialice.
Desventajas	Se deben realizar ciertas actividades en áreas de proceso que quizás no sean relevantes para la empresa.	Pueden quedar áreas de proceso sin demasiada calidad, debido a que se centran los esfuerzos en unas pocas. La organización como un todo se sigue considerando inmadura, si es que no implementa las actividades que plantea CMMI nivel 1 en las áreas de proceso.
Diferencias	- Niveles: 5 → 1 a 5 - Niveles: indican madurez organizacional - Definidos por un conjunto de áreas de proceso - Provee una secuencia para mejorar determinadas áreas de proceso progresivamente	- Niveles: 6 → 0 a 5 - Niveles: indican capacidad de un área de proceso - Definidos por cada área de proceso - Permite mejorar en un área de proceso que se desea

Relación de CMMI con Ágil CMMI cara a cara con Ágil

- "Nivel 1"
 - Identificar el alcance del trabajo
 - Realizar el trabajo
- "Nivel 2"
 - Política Organizacional para planear y ejecutar
 - Requerimientos, objetivos o planes
 - Recursos adecuados
 - Asignar responsabilidad y autoridad
 - Capacitar a las personas
 - Administración de Configuración para productos de trabajo elegidos
 - Identificar y participar involucrados
 - Monitorear y controlar el plan y tomar acciones correctivas si es necesario
 - Obligatoriamente mantener adherencia a los procesos y QI de productos y/o servicios
 - Revisar y resolver aspectos con el nivel de administración más alto

Referencias:
Verde: Da soporte,
Negro: Neutral,
Rojo: Desigual

No coincide con el monitoreo objetivo pues hace referencia a las auditorías realizadas por agentes externos al equipo. Para poder usar CMMI siendo ágil, ambas deben ceder un poco: Ágil plantea de gestión ágil de requerimientos y no pide que haya un control de qué requerimientos tiene el producto en cada momento de tiempo, pero CMMI sí. De alguna forma se debe tener trazabilidad de los requerimientos en US y demás para CMMI. Así como CMMI acepta no tener el registro de las Daily por ejemplo. Ágil debe ceder Auditorías.

- "Nivel 3"
 - Mantener un proceso definido
 - Medir la performance del proceso
- "Nivel 4"
 - Establecer y mantener objetivos cuantitativos para el proceso
 - Estabilizar la performance para uno o más subprocesos para determinar su habilidad para alcanzar logros
 - Monitorear y controlar el plan y tomar acciones correctivas si es necesario
 - Asignar mejora continua para dar soporte a los objetivos
 - Identificar y corregir causa raíz de los defectos

¿Por qué no coincide a medida que subimos de nivel? CMMI dice que tienes que definir un proceso y que después lo tienes que cumplir (lo que se verifica con las auditorías). En cambio, ágil en ningún momento evalúa que hayas seguido el proceso que definiste. CMMI a partir de nivel 3 o 4 plantea métricas/estadísticas de los proyectos y productos, y en base a la comparación de esas medidas se arma una medida del proceso. Ágil plantea que la experiencia no es extrapolable a otros proyectos.

? ↗

Calidad

Formalmente se puede definir como el cumplimiento de los requerimientos funcionales y de performance explícitamente definidos. Todos los aspectos y características de un producto o servicio que se relacionan con la habilidad de alcanzar las necesidades manifiestas o implícitas. Está relacionado con MIS necesidades y MIS expectativas. Está directamente relacionada con la persona (es subjetiva a quién la analice), las circunstancias, el contexto, la edad en un momento de tiempo dado (puede para mí algo no tener calidad, pero para otra persona sí).

Gestión de calidad

La gestión de calidad del software para los sistemas de software tiene tres intereses fundamentales:

- A nivel de organización, la gestión de calidad se ocupa de establecer un marco de proceso y estándares de organización que conducirán a software de mejor calidad.
- A nivel del proceso, la gestión de calidad implica la aplicación de procesos específicos de calidad y la verificación de que continúen dichos procesos planeados.
- A nivel del proyecto, la gestión de calidad se ocupa también de establecer un plan de calidad para un proyecto.

Aseguramiento de calidad

Es la definición de procesos y estándares que deben conducir a la obtención de productos de alta calidad y, en el proceso de fabricación.

El espíritu del aseguramiento de la calidad del SW es actuar como medida preventiva a diferencia del testing que llega tarde.

El plan de calidad debe establecer las cualidades deseadas de software y describir cómo se valorarán.

Problemas en la calidad

- Atrasos en las entregas -> Relacionado a calidad del proyecto.
- Requerimientos no claros -> Relacionado a calidad del producto.
- Software que no hace lo que debería -> Relacionado a calidad del producto.
- Costos excedidos -> Relacionado a calidad del proyecto.
- Trabajo fuera de hora -> Relacionado a calidad del proyecto.
- Fenómeno 90-90 (90% hecho 90% restante) -> Relacionado a calidad del proyecto.
- No se aplica SCM. -> Relacionado a calidad del producto.

Un software de calidad no solo debería cumplir con los requerimientos, sino que también debería poder entregarse a tiempo, no exceder costos, etc.

Calidad en el desarrollo de Software

Los costos excedidos, retrasos en la entrega, falta de cumplimiento de los compromisos, requerimientos no claros hacen que la percepción de nuestro cliente sea mala. Si el medio no tiene calidad, difícilmente se pueda lograr un producto de calidad.

Para que un SW sea de calidad debe satisfacer:

- Las expectativas del cliente;
- Las expectativas del usuario;
- Las necesidades de la gerencia;
- Las necesidades del equipo de desarrollo y mantenimiento;
- Las expectativas de otros interesados.

Calidad para Quién – Visiones de Calidad

A la calidad se la puede analizar desde distintas perspectivas o visiones:

- **Visión del usuario:** esto tiene que ver con las expectativas que tiene el usuario para con el producto, y si este las satisface. Esta es quizás la más complicada de establecer, porque está en la cabeza de los usuarios, lo cual es una razón más por la que el principio de comunicación constante es crucial, para ir validando en todo momento si estamos en el camino correcto. El agilismo trata de incluir la visión de calidad del usuario a través del rol de Product Owner, el cual es ocupado por una persona con capacidad de decisión del cliente.
- **Visión del producto:** esto se asocia al nivel de satisfacción de los requerimientos particulares de cada producto.
- **Visión del proceso** (manufactura): si el proceso de desarrollo utilizado es el correcto para el producto que se desea desarrollar, es decir, el proceso aporta valor al producto y no produce desperdicios.
- **Visión del valor:** encontrar un equilibrio en la relación costo-beneficio, para obtener siempre el mayor valor para el cliente posible
- **Visión Trascendental:** esta visión es un tanto utópica, ya que se asocia a objetivos difíciles de alcanzar. Por ejemplo 0 defectos en el producto, pero son aquellos objetivos que motivan a seguir en busca de la mejora continua

Hay que insertar tareas para asegurar la calidad del producto. Ahí es donde aparecen varias técnicas como revisiones técnicas (realizado entre pares, con el propósito de detectar tempranamente el defecto), auditorías de configuración funcional (la que valida una versión de la línea base), auditorías de configuración física (la que verifica una versión de la línea base), Testing (es para controlar la calidad cuando el producto ya está listo). Los modelos de calidad dicen: hace tu proceso de manera que tenga calidad para vos (empresa), pero este proceso debe ser compatible con lo que yo te digo (modelo de referencia).

Calidad del producto

Definimos anteriormente que la calidad es el nivel de cumplimiento o adecuación a las necesidades (explícitas e implícitas) y para el caso del producto estas deberían estar explicitadas en los requerimientos. Es por esto que los requerimientos son tan importantes, porque si no están o están mal definidos, no tenemos contra qué validar.

La gestión de calidad en productos son acciones sistemáticas de las organizaciones para lograr calidad, las cuales requieren equilibrio entre:

- Calidad programada: los alcances del producto planificados.
- Calidad realizada: lo que realmente se ha desarrollado del producto.
- Calidad necesaria: mínimas características que el producto debe tener, para que este satisfaga los requerimientos.

Calidad del Proceso

- No existe en la realidad un proceso que sirva para todas las organizaciones y proyectos en general.
- La calidad de proceso nunca es un fin en sí mismo, lo que realmente nos interesa es la calidad de producto.
- Proceso con calidad → Producto con calidad
- Se insiste tanto en la calidad del proceso, debido a que es el único factor controlable para mejorar la calidad del software

Se aplica calidad en el producto y en el proceso. No se habla de aseguramiento de calidad en el proyecto, dado que esta se encuentra implícita por el hecho de que el proyecto implementa un proceso

IMPORTANCIA DE TRABAJAR PARA Y CON LA CALIDAD

Importancia

1. Aumenta la satisfacción del cliente, al entregar software confiable, estable y que cumple con los requerimientos.
2. Reduce costos a largo plazo, porque prevenir errores es más barato que corregirlos en producción.
3. Mejora la reputación de la organización, generando confianza y fidelización de los clientes.
4. Facilita el mantenimiento y la evolución del producto
5. Optimiza los procesos internos, ya que la calidad se basa en la mejora continua y la estandarización.
6. Cumple normas o certificaciones, como ISO 9001 o CMMI.

Ventajas

Descripción:

Menor cantidad de errores y fallas
Procesos más ordenados, menos retrabajo y mejor comunicación entre equipos.
Mejor uso de recursos
Clientes más satisfechos
Facilidad de mantenimiento
Cumplimiento de estándares

Desventajas

Mayor inversión inicial
Resistencia al cambio, Algunos equipos pueden rechazar
Si los controles son excesivos, pueden volver el proceso más lento o rígido.
obtener certificaciones (ISO, CMMI, etc.) puede ser costoso.
Tiempo adicional de planificación

ACTIVIDADES RELACIONADAS CON ASEGURAR CALIDAD

Administración de la calidad de Software

Busca asegurar que se alcancen los niveles requeridos de calidad para el producto de SW, definiendo estándares y proceso de calidad apropiados. El fin en sí de la Administración de Calidad de SW es generar una cultura de calidad y que esta sea vista como una responsabilidad de todos en el equipo.

Entre las actividades que desempeña la administración de calidad, se encuentran:

- Aseguramiento de calidad: define estándares, procesos, procedimientos y modelos de calidad sobre los cuales se van a realizar las comparaciones.
- Planificación de Calidad: se debe planificar la calidad, qué actividades se van a llevar a cabo, cuándo. Se selecciona los estándares aplicables para nuestro proyecto en particular. Esta actividad abarca determinar los productos de SW que se desea que tengan calidad
- Control de Calidad: es la ejecución de lo planificado, y se revisa en qué situación está el proyecto. Asegura que los procedimientos y estándares son respetados por el equipo de desarrollo de SW. Existen dos enfoques para el control de calidad:
 - o Revisiones de calidad: El grupo examina parte de un proceso/producto + documentación, para encontrar potenciales problemas.
 - Tipos de revisiones de calidad:
 - Inspecciones para remoción de defectos (producto);
 - Revisiones para evaluación de progreso (producto y proceso);
 - Revisiones de Calidad (producto y estándares).
 - o Evaluaciones de SW automáticas y mediciones.

Calidad de procesos en la práctica: ¿cómo garantizamos calidad de proceso en la práctica? Cumpliendo con algunas de las siguientes tareas:

- Definir proceso estándares tales como:
 - o Cómo deberían conducirse las revisiones;
 - o Cómo debería realizarse la administración de configuración, etc.
- Monitorear el proceso de desarrollo para asegurar que los estándares sean respetados.
- Reportar en el proceso a la Administración De Proyectos y al responsable del SW.
- No use prácticas inapropiadas simplemente porque se han establecido los estándares.

Tipos de auditoría

SQA (aseguramiento de calidad de sw) ¹

Objetivo: realizar controles de sw y proceso de desarrollo

Asegurar cumplimiento de los estándares y procedimientos

Asegurar q los defectos sean informados para que puedan ser solucionados

Auditorías de calidad de sw

implica una evaluación independiente (realizada por un grupo de trabajo que no tienen nada que ver con el equipo del proyecto) de productos o procesos de software que permiten asegurar el cumplimiento con estándares, lineamientos, especificaciones y procedimientos basados en un criterio objetivo

Las auditorías implican esfuerzo y costo para los proyectos, sin embargo, sus beneficios son superiores.

Son un instrumento para el Aseguramiento de Calidad en el Software.

Beneficios de las auditorías *pg. auditor pg 2*

- Se obtienen opiniones objetivas e independientes;
- Permiten identificar áreas de insatisfacción potenciales del cliente;
- Permiten asegurar que se están cumpliendo con las expectativas;
- Determinan que se implementen de manera efectiva los procesos de desarrollo y actividades de soporte

Beneficios de las auditorías de calidad de software

- Evaluar el cumplimiento del proceso de desarrollo
- Determinar la implementación efectiva de:
 - El proceso de desarrollo organizacional
 - El proceso de desarrollo del proyecto
 - Las actividades de soporte
- Dar visibilidad a la gerencia sobre los procesos de desarrollo



Resultado: Mejores productos conllevan a clientes satisfechos y crecimiento del negocio

Tipos de auditorías

Auditoría de proyecto → valida el cumplimiento del proceso de desarrollo

Responsable de ver que el proyecto se haya ejecutado con el proceso que se definió.

Lo que se hace es tomar evidencias de lo que se está haciendo y contrastarlo con lo documentado en las ERS, arquitectura, diseño, etc.

Acá puede haber diferencias respecto a metodologías tradicionales o empíricas. En SCRUM, por ejemplo, el mismo equipo realiza estas auditorías en la ceremonia de Sprint Retrospective. En cambio, en metodologías tradicionales, el auditor debe ser externo al proyecto y a veces a la empresa. la auditoría se realiza sobre el proyecto.

Estas auditorías se llevan a cabo de acuerdo a lo establecido en las PACS (Plan de Aseguramiento de Calidad de Software).

verifica q el proyecto se haya asegurado en el proceso q se dijo q se iba a seguir... respecto al proceso

Auditorías de Configuración Funcional → valida q el producto cumple con sus req

Valida que el producto cumpla con los requerimientos. La auditoría funcional compara el software que se ha construido (incluyendo sus formas ejecutables y su documentación disponible) con los requerimientos de software especificados en la ERS.

El propósito de la auditoría funcional es asegurar que el código implementa sólo y completamente los requerimientos y las capacidades funcionales descritos en la ERS.

Auditoría de configuración física → valida q el ítem de configuración cumple con la documentación técnica q lo describe

Valida que el ítem de configuración cumpla con la documentación técnica que lo describe

La auditoría física compara el código con la documentación de soporte.

Su propósito es asegurar que la documentación que se entregará es consistente y describe correctamente al código desarrollado.

El PACS debería indicar la persona responsable de realizar la auditoría física.

Auditorías externas de Aseguramiento de Calidad (No sé si son un tipo más)

Son auditorías realizadas por un personal externo a la organización, a quienes se encargan de realizar las auditorías de calidad. Esto permite evaluar a los miembros de QA, para determinar si sus evaluaciones dentro de la organización se están realizando de forma correcta.

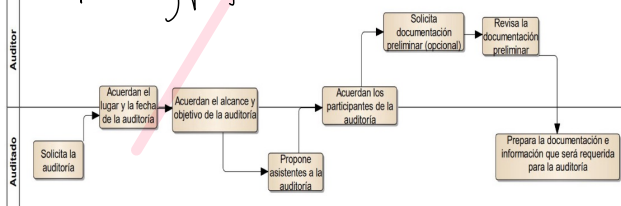
Roles en una auditoría

- **Auditor:** entrega evidencia, contesta las dudas del auditor, propone planes de acción. Generalmente es el Líder de Proyecto, pero puede ser otro. *figura del equipo*
- **Auditor:** puede ser una persona o dos y deben ser de fuera del proyecto que se está auditando. Realiza la auditoría.
- **Gerente de SQA (Software Quality Assurance):** Es quien prepara el plan de auditoría, calcula su costo, asigna recursos, resuelve las no-conformidades entre auditor y auditado. (orientado a la gestión de la auditoría)

Proceso de auditoría

Etapas Auditoría

1. Preparación y planificación

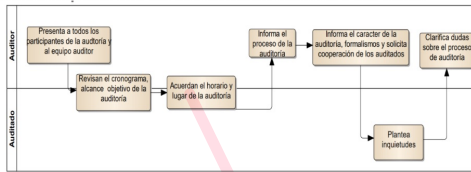


No son sorpresa. El auditado solicita la auditoría y junto con el auditor definen la fecha, el alcance y objetivo, acordando los participantes de esta. Se deben definir métricas de auditoría, como por ejemplo: esfuerzo por auditoría, cantidad de desviaciones, duración de la auditoría, cantidad de desviaciones clasificadas por auditor de CMMI, etc.

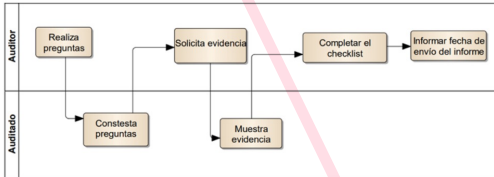
2. Ejecución

El auditor escucha lo que la gente dice que hace y luego ve la documentación, pide evidencia (lo que debería estar haciéndose).

- Reunión de apertura: se informa cómo será el proceso, indicando el lugar y la hora.



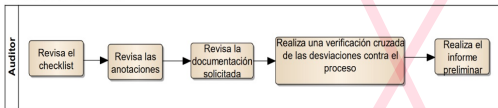
- Ejecución: se responden las preguntas, se muestra la evidencia y se completa el checklist. El auditor escucha lo que la gente dice que hace y luego ve la documentación (lo que debería estar haciéndose).



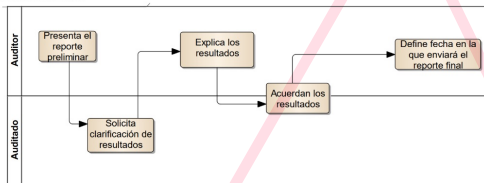
3. Analisis y Reporte de los resultados

El auditor realiza el reporte de resultados y el auditado analiza la respuesta, puede o no estar de acuerdo con algunas prácticas y se deja asentado el documento final. Se evalúan los resultados, se hace una reunión de cierre y se hace entrega del reporte final.

- Evaluación de Resultados



- Reunión de cierre

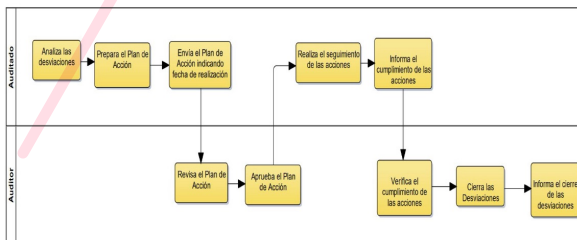


- Entrega de reporte final



4. Seguimiento

Se analizan las desviaciones y prepara un plan de acción que se envía al auditor para que lo revise y apruebe. En función del acuerdo entre auditado y auditor, puede que el auditor haga un seguimiento de las desviaciones que encontró hasta que considere que han sido resueltas.



Herramientas y técnicas utilizadas en auditorías

- Checklists: tienen preguntas tipo para garantizar que independientemente de quién haga la auditoría el foco de las cosas que se controlan sea el mismo. Son de mínima, es decir, se debe garantizar de mínimo contestar estas preguntas, pero el auditor puede solicitar más cosas (hacer más preguntas).
- Muestreo: consiste en seleccionar una muestra representativa de los productos y/o procesos a auditar.
- Revisión de registros
- Herramientas automatizadas

Resultados/hallazgos de una auditoría

- Buenas prácticas: cuando es algo superior a lo definido que tenemos que hacer. Mucho mejor de lo esperado. Ejemplo: se armó una muy buena herramienta para gestionar el plan de proyecto para que la gente pueda accederlo y mejorarlo.
- Desviaciones: cualquier cosa que no se hizo o que no se hizo cómo el proceso lo definió. Hay dos grados:
 - o No Adecuado: lo que realmente está mal.
 - o Necesita Mejora: si lo estás haciendo, pero necesita mejorarse.Requiere un plan de acción por parte del auditado.
- Observaciones: son cosas que se advierten por el auditor que no llegan a ser desviaciones pero que pueden llegar a ser riesgosas las prácticas y pueden llegar a generar un problema, por eso se destacan a consideración del equipo para que ellos decidan si hacerlo o no. Algo para revisar. Deberían mejorarse, pero no requieren un plan de acción. Ejemplo: técnica de estimación mala.

Métricas de auditoría

Cada organización establece según sea apropiada o no:

- Esfuerzo por auditoría.
- Cantidad de desviaciones.
- Duración de auditoría.

- Calidad de Producto:
 - o Planificación de pruebas para el software- Niveles y tipos de pruebas para el software.
 - o Técnicas y herramientas para probar software.
 - o Testing agile
 - o Técnicas y Herramientas para la realización de revisiones técnicas del software.

Calidad de Producto

Testing

- para ver si el SW es válido o no respecto a los req
- es una activ destructiva enfocada en encontrar defecto
- QA todo el ciclo de vida, testing cuando está construido

Definición de Testing

Proceso mediante el cual se somete a un software o un componente de software a condiciones específicas con el fin de determinar y demostrar si el mismo es válido o no en función de los requerimientos especificados.

El testing es una actividad destructiva cuyo objetivo es encontrar defectos, cuya presencia es asumida de antemano en el código.

asegurar que el código hace lo que fue diseñado para hacer.

El Testing es parte del QA (Quality Assurance) y forma parte del control de calidad. Se hace cuando el producto ya está construido (o lo que se desea testear), en cambio el QA se aplica en todo el ciclo de vida

Las actividades de testing se consideran exitosas si se encuentran defectos en la ejecución de los casos de prueba.

El testing es una actividad costosa, por lo que es necesario lograr un nivel de cobertura, teniendo en cuenta el costo-beneficio. Nunca se podrá cubrir el 100% de las pruebas. Esta cobertura se comienza a definir desde el momento en los cuales se establecen los criterios de aceptación.

Principios del Testing

Principios del Testing	
	• Un programador debería evitar probar su propio código. • Una unidad de programación no debería probar sus propios desarrollos. → lo mismo q el primero pero con empresa y no desarmado • Examinar el software para probar que no hace lo que se supone que debería hacer es la mitad de la batalla, la otra mitad es ver que hace lo que no se supone que debería hacer. • No planificar el esfuerzo de testing sobre la suposición de que no se van a encontrar defectos.
El testing muestra presencia de defecto	
El testing exhaustivo es imposible	
Testing temprano → cuanto antes mejor	
Agrupamiento de Defectos → los pruebas deben organizarse y ordenar	
Paradoja del Pesticida → hay q eliminar las plagas de prueba	
El testing es dependiente del contexto	
Falacia de la ausencia de errores	

Artefactos de testing

- plan de prueba
- casos de prueba
- Reporte de defect
- Informe final

Hay q ver q cumple como q se debe hacer y ver q no debe hacer

Defecto vs Error

La diferencia entre ambos conceptos es el momento en el cual se detectan y solucionan los errores o defectos. Un error es detectado y corregido en una misma etapa, y un defecto es un error que se traslada de una etapa a otra etapa posterior en la cual se introdujo. El testing encuentra defectos, ya que son errores que surgieron en etapas anteriores, pero se detectan en la etapa de prueba.

Hay que aclarar que ambos conceptos pueden o no generar fallas en el sistema

Defecto

IMPORTANTE

Un defecto posee dos características principales, que permiten catalogarlos en la etapa de testing:

- Severidad: define la gravedad del defecto, y es determinada por la persona que realiza el testing. El valor de la severidad se asigna dependiendo de la siguiente escala:
 - o Bloqueante → el defecto no permite continuar con la ejecución del sistema.
 - o Crítico → el sistema funciona, pero la funcionalidad que se está testeando tiene un defecto crítico.
 - o Mayor → la funcionalidad que se está testeando funciona, pero no de forma correcta.
 - o Menor → la funcionalidad se ejecuta correctamente, pero con advertencias erróneas o errores de baja importancia.
 - o Cosmética → formato de fechas, formato de números, distribución de componentes en una GUI, temas de presentación de interfaces, etc.
- Prioridad: la prioridad define el impacto del defecto para el negocio, y permite ordenar la atención de los defectos según las necesidades del cliente. Una escala posible para la prioridad puede ser:
 - o Urgencia
 - o Alta
 - o Media
 - o Baja

en las esquinas, los límites están los bugs (hay q tener en cuenta esto)

CASOS DE PRUEBA ES FUNDAMENTAL PARA HACER TESTING

Salvo de los requerimientos para poder escribir esto cuando yo detecto un error debo poder reproducirlo (si no puedo reproducirlo no voy a poder corregirlo) es un caso de paso/condición/entorno q se usan para comprobar q una parte del sist funciona según lo operado

Deben ser Reproducibles

OBJETIVO → descubrir errores
CANTIDAD → en forma completa
ABSTRACCIÓN → en el mínimo de esfuerzo y tiempo

Casos de prueba

Son una secuencia de pasos que hay que seguir para obtener un resultado esperado, y se tienen que dar ciertas condiciones previas que fijan una situación para que se pueda ejecutar la prueba.

Es el artefacto más importante del Testing. Este se hace una vez, pero sirve para realizar infinitas pruebas, debiendo obtener en todas el mismo resultado esperado. Los casos de prueba salen de los requerimientos.

Niveles de prueba

Los niveles de prueba determinan el foco o la granularidad de la prueba que se está por ejecutar.

- o **Pruebas unitarias**: las hace el desarrollador. Son pruebas que se realizan **sobre un componente**. Se hacen teniendo acceso al código fuente, y se pueden usar herramientas para realizarlas
- o **Pruebas de integración**: El propósito de la ejecución de estas pruebas es verificar el funcionamiento de dos o más componentes juntos que se relacionen entre sí. pero no probar el sist como un todo
- o **Pruebas de sistema o de versión**: En estas pruebas se realizan **testeos sobre la versión de un incremento de producto o de un producto** y deben realizarlas **personas ajenas al desarrollo de los programas**
- o **Pruebas de aceptación de usuario**: se realizan en el despliegue y debería realizarlas el usuario para verificar que se cumple con lo que el mismo requirió. Busca que el cliente/usuario se familiarice con el producto

Ambientes de Testing

Son todos los recursos, tanto hardware como software, que se requieren para poder trabajar con el producto. Existen distintos ambientes, los cuales poseen distintas necesidades, según la etapa de desarrollo

Desarrollo desarrollo y pruebas unitarias

Implica hardware y software necesarios (librerías, extensiones, IDEs, compiladores, etc.) para poder **desarrollar y desplegar el producto** y utilizarlo. En este ambiente se realizan las pruebas unitarias

Pruebas trabajo los testers

Es el ambiente que utilizan los testers para llevar a cabo las pruebas, y los desarrolladores no deben tener acceso. En este se realizan las pruebas del sistema. y de integración

Reproducción ya hace datos reales

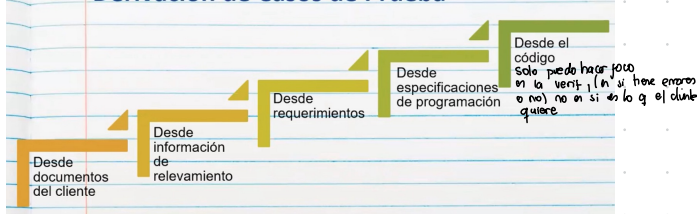
Debería tener las mismas características que el ambiente productivo para poder comprobar que el producto funcionará una vez desplegado en producción de manera correcta. En este ambiente se realizan las pruebas de aceptación.

El problema de este entorno es que muchas veces resulta difícil (o imposible en algunos casos).

Producción

Este entorno, es la configuración de software y hardware que tienen los usuarios finales del software, y que utilizan para el desempeño de sus actividades laborales. Obviamente en este entorno no se realizan pruebas (probar en este ambiente tiene grandes consecuencias), ya que en teoría la versión del producto cumple con los criterios del Definition of Done.

Derivación de Casos de Prueba



Proceso de pruebas

El Testing es un proceso definido (que se lleva a cabo durante todo el ciclo de vida del producto); está compuesto de etapas detalladas.

planificar pruebas como un proceso

- **Planificación**: Determina como se incluirá el Testing en el plan del proyecto, y como será el Test Plan. El resultado de la planificación es el Plan de Pruebas, que debe contener:
 - o Riesgos y objetivos del Testing.
 - o Estrategia de Testing.
 - o Recursos.
 - o Criterio de Aceptación.
- **Diseño** (Identificación y especificación de casos de prueba): Analiza si los requerimientos son testeables o no.
- **Ejecución**: Cuando se ejecutan los CP, se registran y se comparan los resultados generando un reporte de defectos. Se trata de automatizar lo más que se pueda respecto de estas ejecuciones (ahorrando tiempo/costo).
- **Análisis de fallas** (Evaluación y Reporte): Se hace un seguimiento de la corrección de los defectos encontrados hasta que se cierran todos los CP, es decir que se hayan solucionado todos.
- **Fin de las pruebas**: En las empresas más maduras se deja de probar recién cuando no hay defectos bloqueantes, críticos, mayores y los defectos menores y cosméticos son muy pocos. Se confecciona el informe final (opcional).

Armo un plan sobre como voy a hacer las pruebas

los tipos de pruebas que func o no func

Definir casos de prueba con variables concretas

Casos de prueba (cont)

- Es el artefacto más importante del testing, y consiste en una secuencia de pasos a seguir, las cuales describen un conjunto de acciones a realizar para lograr un resultado concreto y esperado. Para esto se deben explicitar Casos de prueba las condiciones de inicio (las cuales fijan una situación particular), y los datos utilizados en ese escenario.
- La característica más importante de los casos de prueba es ser **REPRODUCIBLES**. Si se encuentra un defecto y no es reproducible a través de un caso de prueba, no es un defecto.

ESTRATEGIAS Son complementarias

Se utilizan para pasar por la mayor cantidad de funcionalidades con la menor cantidad de casos pruebas confeccionados, debido a que el tiempo y el presupuesto para diseñarlos y ejecutarlos es limitado.

CAJA BLANCA

No están basadas en especificaciones, analizamos la estructura interna del sw. Se basan en el código

CAJA NEGRA

Están basadas en especificaciones, Se definen entradas y salidas q se esperan. Basada en experiencias, tiene q ver con la experiencia del tester, no se dispone de la estructura interna de la implementación

Se realiza un testing exhaustivo de entrada, probando cada posible entrada conducida como caso de prueba, no necesariamente las válidas.

Se clasifican en basados en especificaciones y basados en experiencia:

- Métodos basados en especificaciones: Son aquellos que se ejecutan utilizando la documentación de especificaciones realizadas del producto.
 - o Partición de equivalencias: proceso sistemático que consiste en identificar clases de equivalencia, que definen subconjuntos de datos que producen un resultado equivalente.
 - o Análisis de los valores limites: variante del anterior. Utilizar los limites o valores de borde de las clases de equivalencia para la definición de los casos de prueba.
- Métodos basados en experiencia: la experiencia y los conocimientos del tester son fundamentales para determinar las entradas del sistema y analizar los resultados.
 - o Adivinanza de defectos: enfoque basado en la intuición y experiencia
 - o Testing exploratorio: el tester mientras va probando el software

Partición de equivalencias

Analiza las condiciones externas (entradas y salidas) involucradas en una funcionalidad determinada. A esas condiciones externas las divide en clases de equivalencia, las cuales son subconjuntos de valores posibles, que arrojan resultados equivalentes en la funcionalidad que se quiere probar.

Ejemplos de entradas: campos de texto, fechas, coordenadas de un mapa, archivos, señales de sensores, etc.

Ejemplos de salidas: mensaje en pantalla, advertencias, listados, tablas, emisión de señal, etc.

Ciclos de prueba → *ejecutar los casos de prueba q establezq termine un ciclo de prueba*

Es la ejecución de un conjunto de casos de prueba en una versión determinada del producto. Generalmente se tienen 2 ciclos, y el primero es conocido como ciclo 0. El ciclo 0 siempre es manual, es donde se configura todo y a partir del ciclo 1 ya se pueden automatizar las pruebas. En caso de detectarse defectos, el producto vuelve a desarrollo para su corrección. Si existe una cantidad alta de ciclos de prueba, es probable que QA no sea bueno, por lo que deberían revisarse ciertos aspectos acerca de la calidad del proceso y producto

Se define cuando liberar el producto con el cliente

Ciclo de pruebas con regresión

Se debe realizar una verificación de la nueva versión, para q no se introduzcan nuevos defectos ya q al solucionar defectos suelen surgir nuevos. una vez q hiciste arreglos deberías volver a probar todo el sistema de juego, y si hay nuevos poder corregirlos

Tipos de prueba

Smoke Test → *primer comida (barrida) catastrófica*

Es un tipo de prueba que se hace para validar que no haya fallas groseras, de gran magnitud, en el producto de software. Se realiza antes de comenzar el ciclo 0.

Testing Funcional

Controla que el software se comporte de la misma manera que lo especificado en la documentación, cumpliendo con las funcionalidades y características definidas. Hay testing funcional basado en dos aspectos:

- Basado en requerimientos: cuando se prueban requerimientos específicos, apunta a probar una funcionalidad sola
- Basado en proceso de negocio: cuando se prueba un proceso de negocio completo, es decir, se prueba todo el proceso.

Testing No Funcional

Se basa en cómo trabaja el sistema haciendo foco en los requerimientos no funcionales (foco en el "cómo", no en el "qué"). Incluye varios tipos de prueba

cuando el producto se despliega en producción

Test Driven Development – TDD → *si el desarrollador sabe de antemano como se va a probar, va a hacer el código con menos defectos*

Este es un tipo de proceso en el que nos enfocamos en construir primero la prueba unitaria cuando tengo los requerimientos y luego codear el componente. Si pienso en las pruebas después tengo menos errores

Es una técnica de desarrollo del software que involucra dos prácticas:

1. Test first development: en esta técnica primero se escriben las pruebas unitarias referentes a la característica de producto a implementar. Definidas las pruebas unitarias, se piensa en la codificación necesaria para que las pruebas se ejecuten con éxito.
2. Refactoring: esta técnica consiste en reestructurar el código existente sin modificar el comportamiento que el mismo provee. Implica la mejora de aspectos no funcionales del software

9 Principios para Testing en Proyectos Ágiles

Testing se mueve hacia adelante en el proyecto

Testing no es una fase

Todos hacen testing

Reducir la latencia del feedback

Las pruebas representan expectativas

Mantener el código limpio, corregir los defectos rápido

Reducir la sobrecarga de documentación de las pruebas

Las pruebas son parte del "done"

De probar al final a Conducido por Pruebas

- Validación: ¿Estamos construyendo el producto correcto?
- Verificación: ¿Estamos construyendo el producto correctamente?

- Principios
- Prevenir es mejor que curar.
 - Evitar es más efectivo que eliminar.
 - La retroalimentación enseña efectivamente.
 - Priorizar lo rentable.

- Olvidarse de la perfección, no se puede conseguir.
- Enseñar a pescar, en lugar de dar el pescado.

REVISIONES TÉCNICAS

Revisiones técnicas – Peer Review

Es una actividad realizada por un colega, cuyo propósito es mejorar la calidad de software, mediante la detección temprana de errores en cualquier artefacto que se genere, por ejemplo, en el código, requerimientos, diseño, arquitectura, riesgos, estimaciones, planes, etc.

Es un proceso estático de validación y verificación; y no corrige errores.

Objetivo: introducir el concepto de verificación y validación. Busca evitar el retrabajo. Motiva a realizar un mejor trabajo.

Ventajas:

- Pueden descubrirse muchos errores;
- Pueden inspeccionarse versiones incompletas;
- Pueden considerarse otros atributos de calidad.

Desventajas:

- Es difícil introducir las inspecciones formales;
- Sobrecargan al inicio los costos y conducen a un ahorro sólo después de que los equipos adquieran experiencia en su uso;
- Requieren tiempo para organizarse y "parecen" ralentizar el proceso de desarrollo.

Tipos de revisiones

- Formales: tienen un proceso definido con roles.
Inspecciones (Inspección de código de Fagan e inspección de Gilb).
- Informales: cuando no existe un proceso de cómo realizarlo.
Walkthrough o recorrido.

Walkthrough o recorrido (informal)

→ se sienta al lado y te dice q puedes mejorar

Técnica de análisis estático donde los participantes formulan preguntas y realizan comentarios acerca de posibles errores, violación de estándares de desarrollo y otros problemas.

No existe un proceso formal. Consiste en reuniones informales de colegas donde se debaten las correcciones a aplicar al producto de trabajo.

Tiene los siguientes objetivos:

1. Mínima Sobrecarga
2. Capacitación de Desarrolladores
3. Rápido retorno

Esta técnica no obtiene métricas para aprender y dejar registros. Sin embargo, es una de las técnicas más elegidas en los enfoques ágiles, después de completar cada iteración del software (una revisión rápida)

Inspecciones (formal) → no busca solucionar errores ni evaluar a la gente

Tiene un proceso formal y cuenta con un conjunto de roles. Es necesario la utilización de un checklist, que ayuda a la memoria para saber que cosas controlar. Se toman métricas y finalmente se realiza un reporte de la revisión al final de la inspección para analizar los defectos encontrados.

Es una actividad que garantiza la calidad del software, cuyo éxito depende de la planificación. Tiene como objetivos:

1. Descubrir errores.
2. Verificar que el software alcanza sus requerimientos.
3. Garantizar que el software fue construido de acuerdo con ciertos estándares.
4. Conseguir un software desarrollado de manera uniforme.
5. Hacer que los proyectos sean más manejables.

Roles participantes

Al ser una técnica formal, si o si debe contar con los siguientes roles:

1. **Autor:** Inicia el proceso seleccionando a un moderador y junto a este eligen al resto de los roles. Entrega el producto a ser inspeccionado al moderador.
 2. **Moderador:** planifica y lidera la revisión. Trabaja junto al autor para elegir los demás roles. Entrega el producto a inspeccionar al inspector 2 días antes de la reunión. Coordina la reunión de forma tal que no ocurran conductas inapropiadas y realiza un seguimiento de los defectos encontrados.
 3. **Anotador:** registra los hallazgos de la inspección.
 4. **Lector:** lee el producto a ser inspeccionado. Este rol es necesario para que los participantes no se dispersen.
 5. **Inspector:** Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación. todos pueden ser inspectores. Todos pueden inspeccionar.
- Ciertos roles pueden ser asumidos por la misma persona.

SON	NO SON
• La forma más barata y efectiva de encontrar fallas • Una forma de proveer métricas al proyecto • Una buena forma de proveer conocimiento cruzado • Una buena forma de promover el trabajo en grupo • Un método probado para mejorar la calidad del producto	• Utilizadas para encontrar soluciones a las fallas • Usadas para obtener la aprobación de un producto de trabajo → no es auditoria • Usadas para evaluar el desempeño de las personas

Etapas del proceso de inspección

- 1. Planificación: el moderador, a pedido del autor, planifica la inspección definiendo el lugar, el tiempo de duración y los roles.
- 2. Visión general: esta etapa es opcional. El autor realiza una descripción general del producto a inspeccionar.
- 3. Preparación: es la preparación de cada rol para la reunión. Cada rol adquiere una copia del producto de trabajo que deberá leer y analizar, con el fin de encontrar potenciales defectos.
- 4. Reunión de inspección: el equipo realiza un análisis para recolectar los potenciales defectos previos y descartar falsos positivos.
- 5. Corrección: finalizada la reunión, el autor realiza las correcciones de los defectos encontradas.
- 6. Seguimiento: Dependiendo de la gravedad puede existir un proceso de re-inspección. De lo contrario, si el defecto era muy simple, el autor simplemente lo corrige.

Cuadro comparativo Auditorías vs. Revisiones técnicas

Aspecto	Auditoría	Revisión
Para qué sirven	Para revelar qué se está haciendo en la realidad, contrastado con lo que se comprometió a hacer.	Encontrar errores en un artefacto lo antes posible, para evitar su propagación.
Roles	Auditor, auditado y gerente de calidad	Autor, moderador, anotador, lector, inspector
Beneficios	Opiniones independientes, identificar áreas de insatisfacción potencial para el cliente, asegurar que se cumplen expectativas, dar visibilidad a procesos de trabajo, etc.	Detección de errores, evitar propagación de defectos, reducción de retrabajo (costos), aumentar calidad de un artefacto, etc.
Etapas	1. Planificación 2. Ejecución 3. Análisis y Reporte de resultados 4. Seguimiento	En revisiones informales no hay pasos a seguir. En revisiones formales: 1. Planificación 2. Visión general 3. Preparación 4. Reunión de Inspección 5. Corrección 6. Seguimiento
Tipos	De proyecto, de calidad, de producto (física y funcional)	Formales e informales
Hallazgos	Buenas prácticas, observaciones y desviaciones	Errores

Respuesta a pregunta de parcial

"Realice un análisis de los principios Lean y explique las prácticas que propone Kanban para aplicar en forma concreta los principios."

- Eliminar desperdicios
 - Visualizar el trabajo (permite visualizar el flujo de trabajo y exponer aquellas actividades que no aportan valor y aquellas que generan tiempos de espera que "traban" el flujo de trabajo)
 - Limitar el WIP (el trabajo a medias genera retrabajo, lo cual es un desperdicio. Al limitar el WIP, se evita ese trabajo a medias para concentrar esfuerzos)
- Amplificar el aprendizaje
 - Hacer explícitas las políticas (que todos los miembros tengan acceso a las políticas usadas para cada unidad de trabajo, actividades a realizar, información necesaria del dominio, etc.)
 - Evolución y mejora continua (no introducir cambios grandes, sino que visualizar el proceso de trabajo actual y aplicar mejoras en él)
- Embeber la integridad conceptual
- Postergar decisiones hasta el momento responsable
 - Limitar el WIP (particularmente el WIP de la columna backlog)
- Ver el todo
 - Visualizar el trabajo (a través del tablero permite que todos los miembros observen todo el trabajo que se hace, el sentido de cada una de las tareas y como estas conforman el flujo de trabajo como un todo que desencadena en la satisfacción de los clientes)
- Dar poder al equipo
 - Haciendo explícitas las políticas y conformando un sistema de trabajo pull, donde cada miembro toma el trabajo cuando esté en condiciones de realizarlo, y no se lo impone nadie.
- Mejorar colaborativamente
- Entregar lo antes posible
 - Gestionar el flujo de trabajo (permite ver de principio a fin los pasos que se deben seguir para lograr cumplir con las necesidades del cliente y hacer aquellos ajustes que sean necesarios para entregar el mayor valor posible, en el menor tiempo posible)